

Claude Code

How it works and how to use it
A practical guide for developers

Covers	What Claude Code is, developer requirements, install and first run, the agentic loop, daily workflow, project configuration, extensions, automation, and a command cheat sheet
Built from	The roadmap.sh Claude Code roadmap and the official Claude Code documentation (code.claude.com/docs)
Prepared	20 September 2026
Version note	Claude Code ships often. Confirm your version with <code>claude --version</code> and check <code>/help</code> and the release notes for anything newer than this guide.

Contents

- 1. What Claude Code is 4**
 - Where you can use it 4
 - What developers use it for 4
- 2. Developer requirements 5**
 - Optional and situational dependencies 5
 - Windows: native or WSL? 5
- 3. Install and first run 6**
 - Install 6
 - Log in and start 6
 - Keeping it current 6
 - Your first 15 minutes 6
- 4. How Claude Code works 8**
 - The agentic loop 8
 - Built-in tools 8
 - What Claude can access 8
 - Where the code runs 8
 - Sessions 9
 - The context window: your scarcest resource 9
 - Safety: checkpoints and permissions 9
- 5. Your everyday workflow 11**
 - Explore, plan, implement, commit 11
 - Give Claude a way to verify its work 11
 - Give precise context 11
 - Steer, do not restart 11
 - Use it to learn a codebase 12
- 6. Configure your project 13**
 - CLAUDE.md: persistent project instructions 13
 - Where configuration lives 13
- 7. Extending Claude Code 14**
 - Skills 14
 - Subagents 14
 - Hooks 15

MCP and plugins	15
8. Automate and scale	16
Non-interactive mode	16
Fan out across many files	16
Run sessions in parallel	16
CI, review and scheduling	16
9. Cheat sheet	18
Commands inside a session	18
Terminal commands and flags	18
Keyboard shortcuts	19
10. Common failure patterns	20
Troubleshooting quick list	20
Security habits	20
11. A learning path	20
Sources and notes	21

1. What Claude Code is

Claude Code is an agentic coding tool. It reads your codebase, edits files, runs commands, and works with your development tools. You describe what you want in plain language; it plans the approach, changes code across multiple files, and checks its own work. It is not limited to the file you have open, which is the main difference from inline code assistants.

Because it can run anything you could run from a command line, it also handles work beyond writing code: running builds and tests, searching a repo, writing docs, managing git, and researching a problem.

Where you can use it

Surface	What it is	Best when
Terminal CLI	The full-featured version. Run <code>claude</code> inside any project directory.	You live in the terminal, script things, or run it in CI.
VS Code / Cursor	Extension with inline diffs, @-mentions, plan review and history.	You want changes shown inside the editor.
JetBrains	Plugin for IntelliJ IDEA, PyCharm, WebStorm and others. Needs the CLI installed separately.	Your team uses JetBrains IDEs.
Desktop app	Standalone app: visual diffs, parallel sessions, scheduled tasks, cloud sessions. Requires a paid subscription.	You prefer a GUI or run many sessions side by side.
Web / mobile	<code>claude.ai/code</code> and the Claude mobile app. No local setup.	Long tasks you check on later, or repos you do not have locally.

One engine, many surfaces
Every surface runs the same underlying engine, so your repo's `CLAUDE.md` files, settings and MCP servers work across all of them.

What developers use it for

- **Clearing the backlog of tedious work:** writing tests for untested code, fixing lint errors project-wide, resolving merge conflicts, updating dependencies, drafting release notes.
- **Building features and fixing bugs:** describe the goal or paste the error; it traces the cause and implements a fix across files.
- **Git and pull requests:** stage changes, write commit messages, create branches, open PRs.
- **Connecting tools with MCP:** Jira, Slack, Google Drive, databases, or your own tooling.
- **Automating with the CLI:** pipe logs in, run in CI, chain with other commands.
- **Running work in parallel or on a schedule:** multiple agents, background sessions, recurring tasks.

Claude Code vs an IDE assistant
The roadmap.sh FAQ frames the choice this way: Claude Code suits execution-heavy work where delegation and automation matter, while an editor-centred tool such as Cursor suits interactive work where fast feedback and visibility into each change matter. Many developers use both.

2. Developer requirements

Installing Claude Code is free. Using it requires a supported account. Check every row below before you start.

Requirement	Details
Operating system	macOS 13.0+, Windows 10 (1809+) or Windows Server 2019+, Ubuntu 20.04+, Debian 10+, Alpine Linux 3.19+
Hardware	4 GB+ RAM; x64 or ARM64 processor
Network	Internet connection required (see the network configuration docs if you are behind a proxy or firewall)
Shell	Bash, Zsh, PowerShell, or CMD
Location	A country on Anthropic's supported-countries list
Account	Claude Pro, Max, Team or Enterprise, or an Anthropic Console account. The free claude.ai plan does not include Claude Code. Amazon Bedrock, Google Cloud's Agent Platform and Microsoft Foundry are supported as third-party providers.

Optional and situational dependencies

Item	When you need it
Git for Windows	Recommended on native Windows so Claude Code can use the Bash tool. Without it, Claude Code uses PowerShell as its shell tool. WSL does not need it.
Node.js 22+	Only if you install through npm. The npm package downloads a native binary, so older Node versions print a warning rather than fail.
ripgrep	Bundled in most installs. On Alpine, install <code>libgcc</code> , <code>libstdc++</code> and <code>ripgrep</code> , then set <code>USE_BUILTIN_RIPGREP=0</code> .
git and gh CLI	Not required to install, but they make commits, branches and pull-request workflows much smoother. Claude can use <code>gh</code> directly.
Project test/build tooling	Claude verifies its work by running your tests, linters and build. The better these commands work, the better the results.

Windows: native or WSL?

Option	Requires	Sandboxing	Use when
Native Windows	Nothing (Git for Windows optional)	Not supported	Windows-native projects and tools
WSL 2	WSL 2 enabled	Supported	Linux toolchains or sandboxed command execution
WSL 1	WSL 1 enabled	Not supported	WSL 2 is unavailable

Before rolling out

Working on a team? Decide up front which plan or provider your organization uses, who configures shared permission rules, and whether to pin a release channel (see section 3).

3. Install and first run

Install

The native installer is recommended. It needs no dependencies and updates itself in the background.

macOS, Linux, WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

Other supported methods:

Method	Command	Updates
Homebrew	<code>brew install --cask claude-code</code> (stable) or <code>claude-code@latest</code>	Manual: <code>brew upgrade claude-code</code>
WinGet	<code>winget install Anthropic.ClaudeCode</code>	Manual: <code>winget upgrade Anthropic.ClaudeCode</code>
apt / dnf / apk	Signed repositories for Debian, Ubuntu, Fedora, RHEL, Alpine (see setup docs)	Through your system upgrade workflow
npm	<code>npm install -g @anthropic-ai/claude-code</code> (never with sudo)	<code>npm install -g @anthropic-ai/claude-code@latest</code>

Log in and start

```
cd your-project
claude                # first run opens your browser to sign in
claude --version      # confirm the install, e.g. 2.1.x (Claude Code)
claude doctor         # read-only diagnostics for install and settings
```

If the `ANTHROPIC_API_KEY` environment variable is set, Claude Code asks you to approve the key instead of opening a browser. That is the usual route for headless machines and CI.

Keeping it current

- Native installs update in the background. Run `claude update` to apply an update immediately.
- Choose a release channel with the `autoUpdatesChannel` setting: `"latest"` (default) or `"stable"` (about a week behind, skipping releases with major regressions).
- Organizations can enforce a floor with `minimumVersion` or require a version range through managed settings.
- Set `DISABLE_AUTOUPDATER` to `"1"` in the `env` block of `settings.json` to stop background checks.

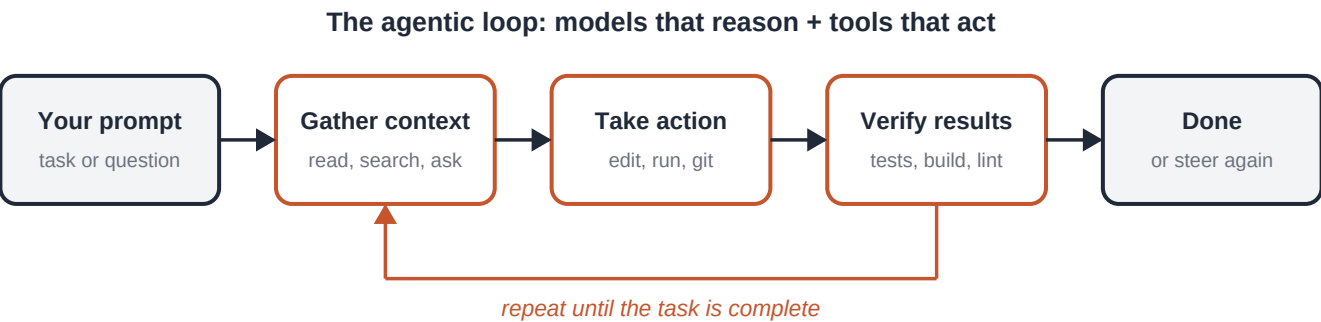
Your first 15 minutes

Step	What to do	Why
1	Ask a read-only question: <code>Explain the architecture of this project</code>	Builds trust and shows how Claude explores your code.
2	Run <code>/init</code>	Generates a starter <code>CLAUDE.md</code> from your project structure.
3	Give one small task with a check: <code>add a unit test for X, run it, fix failures</code>	Shows the loop: edit, run, verify, iterate.
4	Run <code>/diff</code> and read the changes	You stay the reviewer of everything that lands.
5	Ask it to commit with a descriptive message	Completes a full change end to end.

4. How Claude Code works

The agentic loop

When you give Claude a task it works through three blended phases: **gather context**, **take action**, and **verify results**. It uses tools throughout, and each result feeds the next decision. A question about your code may only need the first phase. A bug fix cycles through all three repeatedly.



You are part of the loop: press Esc to stop, or type a correction at any point.

Two components power the loop. **Models** reason about your code and decide what to do next. **Tools** let Claude act: read, edit, search, run, and fetch. Claude Code is the **agentic harness** around the model, supplying the tools, context management and execution environment.

Sonnet handles most coding tasks well; Opus gives stronger reasoning for complex architectural decisions. Switch with `/model` in a session or start with `claude --model <name>`.

Built-in tools

Category	What Claude can do
File operations	Read files, edit code, create new files, rename and reorganize
Search	Find files by pattern, search contents with regex, explore a codebase
Execution	Run shell commands, start servers, run tests, use git
Web	Search the web, fetch documentation, look up error messages
Code intelligence	See type errors after edits, jump to definitions, find references (needs a code intelligence plugin)

Tool use in practice. For "fix the failing tests", Claude typically runs the suite, reads the errors, searches for the relevant source, reads it, edits, and runs the tests again. You never specify those steps.

What Claude can access

- **Your project:** files in the working directory and subdirectories, plus other files with permission.
- **Your terminal:** any command you could run yourself: build tools, git, package managers, scripts.
- **Your git state:** current branch, uncommitted changes, recent history.
- **CLAUDE.md and auto memory:** persistent instructions you write, and learnings Claude saves itself.
- **Extensions you configure:** MCP servers, skills, subagents, and Claude in Chrome.

Where the code runs

Environment	Where code runs	Use case
Local	Your machine	Default. Full access to your files, tools and environment.
Cloud	Anthropic-managed VMs, or self-hosted environments your organization operates	Offload tasks; work on repos you do not have locally.
Remote Control	Your machine, driven from a browser	Use the web UI while files and execution stay local.

Sessions

- Each conversation is saved locally as a plaintext JSONL file under `~/.claude/projects/`. That enables rewinding, resuming and forking.
- Sessions are independent.** A new session starts with a fresh context window. Persist knowledge in `CLAUDE.md` or let auto memory capture it.
- Resume with `claude --continue` (latest) or `claude --resume` (pick from a list). Fork with `--fork-session` or `/branch` to try a different direction without touching the original.
- Sessions are tied to a directory. Use git worktrees to run parallel sessions on different branches.

The context window: your scarcest resource

The context window holds the conversation, every file Claude reads, command output, `CLAUDE.md`, auto memory, loaded skills and system instructions. It fills fast, and performance degrades as it fills. Most best practices come from managing this one constraint.

Tool	What it does
<code>/context</code>	Shows what is using space as a colored grid, with suggestions
<code>/compact [focus]</code>	Summarizes the conversation to free space, e.g. <code>/compact focus on the API changes</code>
<code>/clear</code>	Starts fresh. Use between unrelated tasks.
<code>/btw <question></code>	Side question whose answer never enters the conversation history
Subagents	Work in their own context window and return only a summary
Skills	Load on demand: descriptions at start, full content only when used

Automatic compaction clears older tool output first, then summarizes. Early detailed instructions can be lost, so put persistent rules in `CLAUDE.md` rather than relying on chat history.

Safety: checkpoints and permissions

Checkpoints snapshot files before Claude edits them. Press `Esc` twice or run `/rewind` to restore code, conversation, or both. Checkpoints only cover changes made through Claude's file editing tools, not shell commands or remote systems, and they do not replace git.

Permission modes control what Claude may do without asking. Press `Shift+Tab` to cycle:

Mode	Behavior
Auto	A classifier reviews most actions in the background and blocks risky ones instead of asking. The built-in starting mode on Pro, Max and Team plans for interactive terminal and VS Code.
Manual	Claude asks before file edits and shell commands. Starting mode on other plans.
Accept edits	Edits files and runs common filesystem commands (<code>mkdir</code> , <code>mv</code>) without asking; still asks for other commands.

Mode	Behavior
Plan	Claude explores and proposes a plan without editing source files.

Pre-approve trusted commands such as `npm test` or `git status` with `/permissions` or `.claude/settings.json`, and use `/sandbox` for OS-level isolation of filesystem and network access.

5. Your everyday workflow

Explore, plan, implement, commit

Letting Claude jump straight to code can solve the wrong problem. For anything uncertain or multi-file, separate research from execution with plan mode.

Phase	What you do	Example prompt
Explore	Enter plan mode: Shift+Tab until the status bar shows plan mode, or claude --permission-mode plan	read /src/auth and understand how we handle sessions and login
Plan	Ask for a plan. Press Ctrl+G to edit it in your editor first.	I want to add Google OAuth. What files change? Create a plan.
Implement	Approve the plan or leave plan mode. Ask for tests and a run.	implement the OAuth flow from your plan, write tests, run them, fix failures
Commit	Ask for a commit and PR.	commit with a descriptive message and open a PR

Skip planning when you could describe the diff in one sentence: a typo, a log line, a rename.

Give Claude a way to verify its work

This is the highest-leverage habit. Without a check it can run, "looks done" is the only signal Claude has, and you become the verification loop. Give it something with a pass/fail result: tests, a build, a linter, a script that diffs output, or a screenshot to compare.

Instead of	Try
implement a function that validates email addresses	write validateEmail. Cases: user@example.com is true, invalid is false, user@.com is false. Run the tests after implementing.
make the dashboard look better	[paste screenshot] implement this design, screenshot the result, list differences and fix them
the build is failing	the build fails with [paste error]. Fix it, verify the build passes, address the root cause, do not suppress the error
fix the login bug	users report login fails after session timeout. Check token refresh in src/auth/. Write a failing test that reproduces it, then fix it

Stronger gates for unattended work: set a **/goal** condition so Claude keeps going until it is met; add a **Stop** hook that runs your check and blocks the turn from ending until it passes; or use a separate reviewer subagent so the agent doing the work is not the one grading it.

Give precise context

- **Reference files with @** instead of describing where code lives.
- **Paste images** (screenshots, mockups, diagrams) directly into the prompt.
- **Give URLs** for docs and API references; allowlist frequent domains with **/permissions**.
- **Pipe data in:** **cat error.log | claude**.
- **Point at existing patterns:** "look at HotDogWidget.php and follow that pattern".
- **Scope the task:** which file, which scenario, what testing style (for example, "avoid mocks").

Steer, do not restart

- **Esc** stops Claude immediately; context is preserved so you can redirect.

- Type a correction and press **Enter** to have Claude read it after the current action finishes.
- **Esc Esc** or **/rewind** restores earlier state. "Undo that" also works.
- After two failed corrections on the same issue, run **/clear** and start over with a sharper prompt. A clean session with a better prompt usually beats a long one full of corrections.
- **Delegate, do not dictate.** Give context and the goal, for example "the checkout flow breaks for users with expired cards; the code is in src/payments/; investigate and fix".

Use it to learn a codebase

Ask what you would ask a senior engineer: How does logging work? How do I add an API endpoint? Why does this call `foo()` instead of `bar()`? It is an effective onboarding tool and needs no special prompting. For larger features, ask Claude to interview you first (it has an **AskUserQuestion** tool) and write the result to a **SPEC.md**, then execute that spec in a fresh session.

6. Configure your project

CLAUDE.md: persistent project instructions

CLAUDE.md is a markdown file Claude reads at the start of every session. Put in it what Claude cannot infer from the code. Run `/init` to generate a starter, then refine it, and check it into git so the team improves it together.

```
# Code style
- Use ES modules (import/export), not CommonJS (require)
- Destructure imports when possible

# Workflow
- Typecheck after a series of code changes
- Prefer running single tests, not the whole suite, for speed
- Branch names: feature/<ticket>-<slug>. PRs need a test plan.
```

Include	Exclude
Commands Claude cannot guess	Anything Claude can learn by reading the code
Style rules that differ from defaults	Standard language conventions
Test instructions and preferred runners	Long API documentation (link to it instead)
Repo etiquette: branches, PR rules	Information that changes frequently
Project-specific architecture decisions	File-by-file codebase descriptions
Environment quirks and required env vars	Vague advice like "write clean code"

- Keep it short (under about 200 lines). For each line ask: would removing this cause a mistake? If not, cut it. Long files cause Claude to ignore rules.
- Import other files with `@path/to/file`. Scope rules to file paths with `.claude/rules/`.
- Level-by-level files are additive: user, project, and nested subdirectory files all contribute.
- If Claude keeps skipping one rule, add emphasis such as **IMPORTANT** to that line only.
- Existing **AGENTS.md**? Claude Code can read it alone or alongside **CLAUDE.md**.
- Auto memory also saves learnings across sessions. Manage it with `/memory`.

Where configuration lives

Location	Scope	Shared?
<code>~/.claude/settings.json</code>	All your projects	No, local to your machine
<code>.claude/settings.json</code>	One project	Yes, commit it to the repo
<code>.claude/settings.local.json</code>	One project, personal	No, gitignored
Managed policy settings	Organization-wide	Yes, admin controlled
<code>.mcp.json</code>	Project MCP servers	Yes, if committed
<code>.claude/skills/</code> , <code>.claude/agents/</code>	Project skills and subagents	Yes, commit them

Open the settings UI with `/config`. Reduce approval prompts by pre-approving safe commands with `/permissions`, or run `/fewer-permission-prompts`, which scans your transcripts and proposes a read-only allowlist.

7. Extending Claude Code

You do not need to configure everything on day one. Add each extension when a trigger appears.

Feature	What it does	Use it when
CLAUDE.md	Always-loaded context	Claude gets a convention wrong twice
Skill	Reusable knowledge or workflow, loaded on demand; invoke with <code>/name</code>	You paste the same playbook a third time
MCP server	Connects Claude to external tools and data	You copy data from a system Claude cannot see
Subagent	Isolated worker with its own context; returns a summary	A side task floods your conversation with output
Hook	Runs your script on a lifecycle event, every time	Something must happen without exception
Plugin	Bundles skills, hooks, subagents and MCP servers to install and share	A second repo needs the same setup

Skills

A skill is a folder with a **SKILL.md**. Claude applies it automatically when relevant, or you invoke it with a slash command. Set `disable-model-invocation: true` for workflows with side effects so only you can trigger them.

```
# .claude/skills/fix-issue/SKILL.md
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---
Analyze and fix the GitHub issue: $ARGUMENTS.

1. Use `gh issue view` to get the details
2. Search the codebase for relevant files
3. Implement the fix, then write and run tests
4. Ensure lint and type checks pass
5. Commit with a descriptive message, push, and open a PR
```

Run it with `/fix-issue 1234`. Bundled skills include `/code-review`, `/batch`, `/debug`, `/loop` and `/doctor`.

Subagents

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: opus
---
You are a senior security engineer. Review for injection flaws, auth
problems, secrets in code and insecure data handling. Give line
references and suggested fixes.
```

Ask for them explicitly: "Use a subagent to review this code for security issues." Subagents are also the right tool for open-ended investigation, because the files they read never enter your main context.

Hooks

Instructions in **CLAUDE.md** are advisory. Hooks are deterministic: they run every time their event fires. This example formats every file Claude edits:

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write" }
        ]
      }
    ]
  }
}
```

- A **PreToolUse** hook that exits with code 2 blocks the action and sends your stderr message back to Claude. That is how you enforce rules like "never edit **.env**".
- Useful events: **SessionStart**, **UserPromptSubmit**, **PreToolUse**, **PostToolUse**, **Notification**, **Stop**, **PreCompact**, **SessionEnd**. Browse what is configured with **/hooks**.
- Hook types: **command**, **http**, **mcp_tool**, **prompt** and (experimental) **agent**.

MCP and plugins

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
/mcp # inside a session: connection status and OAuth
```

MCP tool definitions are deferred by default, so idle servers cost little context. Run **/plugin** to browse plugins; for typed languages, install a code intelligence plugin so Claude sees type errors after every edit. Prefer CLI tools such as **gh**, **aws** or **gcloud** where they exist: they are the most context-efficient way to reach external services.

8. Automate and scale

Non-interactive mode

`claude -p "prompt"` runs once and exits. Use it in scripts, pre-commit hooks and CI.

```
claude -p "Explain what this project does"
claude -p "List all API endpoints" --output-format json
claude -p "Analyze this log file" --output-format stream-json --verbose
tail -200 app.log | claude -p "Flag any anomalies"
git diff main --name-only | claude -p "Review these files for security issues"
```

Fan out across many files

```
# 1. Ask Claude to write the target list to files.txt, then:
for file in $(cat files.txt); do
  claude -p "Migrate $file from Python 2 to Python 3. Return OK or FAIL." \
    --allowedTools "Edit,Bash(git commit *)"
done
```

Test on two or three files, refine the prompt, then run the rest. `--allowedTools` scopes what Claude may do, which matters when nobody is watching. In a git repo, `/batch <instruction>` does this for you: it splits a change into 5 to 30 units, and each subagent works in its own worktree and opens a pull request.

Run sessions in parallel

Approach	Notes
Git worktrees	Separate directories per branch so edits do not collide
Desktop app	Manage multiple local sessions visually
Cloud sessions	Run on Anthropic-managed infrastructure; pull one back with <code>claude --teleport</code>
Agent view	<code>claude agents</code> dispatches background sessions you watch from one screen (research preview)
Agent teams	Experimental and off by default; shared tasks and a lead agent

Writer / reviewer pattern

Session A implements a feature. Session B, with fresh context, reviews it: "Review the rate limiter in `@src/middleware/rateLimiter.ts` for edge cases and race conditions." A reviewer that did not write the code is less biased toward it. Tell the reviewer to report only gaps that affect correctness or stated requirements, or you will get over-engineering.

CI, review and scheduling

- **GitHub Actions and GitLab CI/CD** automate PR review and issue triage. `/install-github-app` walks through GitHub setup.
- **Auto mode in scripts:** `claude --permission-mode auto -p "fix all lint errors"` lets a classifier block risky actions while routine work proceeds without prompts.
- **Routines** run in the cloud on a schedule or trigger, even when your computer is off. **Desktop scheduled tasks** run locally. `/loop 5m check if the deploy finished` repeats a prompt inside a session.
- **Before shipping:** `/diff`, `/code-review` (add `--fix` to apply findings) and `/security-review`.

- **Agent SDK:** build your own agents on Claude Code's tools, with full control over orchestration, tool access and permissions.

9. Cheat sheet

Commands inside a session

Command	Purpose
<code>/init</code>	Generate a starter CLAUDE.md
<code>/memory</code>	Edit CLAUDE.md files and manage auto memory
<code>/plan [task]</code>	Enter plan mode from the prompt
<code>/model, /effort</code>	Switch model; set reasoning effort
<code>/context, /compact, /clear</code>	See, shrink or reset the context window
<code>/btw <question></code>	Side question that stays out of history
<code>/permissions</code>	Manage allow, ask and deny rules
<code>/mcp, /hooks, /plugin</code>	Manage MCP servers, view hooks, browse plugins
<code>/resume, /branch, /fork</code>	Return to, branch or copy a conversation
<code>/rewind</code>	Roll code and conversation back to a checkpoint
<code>/diff</code>	Review working-tree changes, including Claude's edits
<code>/code-review, /security-review</code>	Review the current diff for bugs or vulnerabilities
<code>/batch <instruction></code>	Large change fanned out across worktree subagents
<code>/goal <condition></code>	Keep working until a condition is met
<code>/loop [interval] [prompt]</code>	Repeat a prompt while the session is open
<code>/doctor</code>	Diagnose and fix setup problems; trims a bloated CLAUDE.md
<code>/config</code>	Open settings (theme, model, output style)
<code>/desktop, /remote-control</code>	Continue this session in the desktop app or from another device
<code>/help</code>	List commands available to you

Terminal commands and flags

Command	Purpose
<code>claude</code>	Start an interactive session in the current directory
<code>claude "query"</code>	Start a session with an initial prompt
<code>claude -p "query"</code>	Run once non-interactively, print the result, exit
<code>claude --continue / --resume</code>	Reopen the latest session / pick from a list
<code>claude --model <name></code>	Choose the model for this session
<code>claude --permission-mode plan auto</code>	Start in a specific permission mode
<code>--output-format json stream-json</code>	Structured output for scripts (add <code>--verbose</code> for streaming)
<code>--allowedTools "..."</code>	Limit which tools may run without asking
<code>claude update / claude doctor</code>	Update now / print diagnostics
<code>claude mcp add ...</code>	Connect an MCP server
<code>claude agents</code>	Watch background sessions from one screen

Keyboard shortcuts

Keys	Action
Esc	Stop Claude mid-action; context is kept
Esc Esc	Open the rewind menu
Shift+Tab	Cycle permission modes
Ctrl+G	Open the plan in your editor
Ctrl+O	Open transcript view (see hook and tool outcomes)
@ then a path	Reference a file in your prompt

10. Common failure patterns

Pattern	What happens	Fix
The kitchen-sink session	Unrelated tasks pile up and the context fills with noise	<code>/clear</code> between unrelated tasks
Correcting over and over	Failed approaches pollute the context	After two misses, <code>/clear</code> and write a better first prompt
The over-long CLAUDE.md	Important rules get lost among trivial ones	Prune hard; turn must-happen rules into hooks; run <code>/doctor</code>
Trust, then verify (too late)	Plausible code that misses edge cases	Always supply tests, scripts or screenshots. If you cannot verify it, do not ship it
Infinite exploration	"Investigate X" with no scope reads hundreds of files	Scope narrowly, or delegate to a subagent

Troubleshooting quick list

- **claude: command not found:** open a new terminal, then run the troubleshooting guide for installation; `claude doctor` reports conflicting or leftover installs.
- **Search fails on Alpine or musl systems:** install `ripgrep` and set `USE_BUILTIN_RIPGREP=0`.
- **Hook does nothing:** check `/hooks`, confirm the matcher's case, and test the script by piping sample JSON into it. Debug with `claude --debug`.
- **Context keeps refilling:** one huge file or tool output is likely the cause. Use `/context` to find it and scope your reads.
- **Windows Git Bash not found:** set `CLAUDE_CODE_GIT_BASH_PATH` in `settings.json`, or run without it and use the PowerShell tool.

Security habits

- Start read-only on unfamiliar or sensitive code, then allow edits.
- Keep secrets out of prompts; protect files like `.env` with a `PreToolUse` hook or deny rules.
- Review the diff before every commit. Claude is fast, but you own the change.
- In shared repos, rely on branch protection and required reviews for generated changes.
- Remember that Bash-made changes are not covered by checkpoints; commit often.

11. A learning path

A staged path from first install to team-wide adoption, following the step-by-step spirit of the `roadmap.sh` guide. Move on when the earlier stage feels routine.

Stage	Learn	Milestone
1. Foundations	Install, sign in, <code>claude</code> , read-only questions, permission modes, <code>/diff</code> , <code>/rewind</code>	You complete a small change and review it yourself
2. Daily driver	Plan mode, verification habits, <code>@</code> references, sessions and resume, <code>/context</code> , <code>/clear</code> , <code>CLAUDE.md</code>	You trust it with multi-file changes, with tests as the gate
3. Customization	Skills, hooks, MCP servers, subagents, plugins, code intelligence, permission allowlists	Your repeated prompts became skills; must-happen rules became hooks
4. Automation	<code>claude -p</code> , output formats, worktrees, <code>/batch</code> , <code>/loop</code> , routines, GitHub Actions or GitLab CI, Agent SDK	Work runs without you watching and is verified before you see it

Stage	Learn	Milestone
5. Team and enterprise	Managed settings, permission policy, release channels and version pins, third-party providers, shared plugin marketplace	A team standard exists and new hires reach stage 2 quickly

Sources and notes

- roadmap.sh Claude Code roadmap and FAQ: <https://roadmap.sh/claude-code>
- Overview: <https://code.claude.com/docs/en/overview>
- How Claude Code works: <https://code.claude.com/docs/en/how-claude-code-works>
- Advanced setup and system requirements: <https://code.claude.com/docs/en/setup>
- Best practices: <https://code.claude.com/docs/en/best-practices>
- Extend Claude Code: <https://code.claude.com/docs/en/features-overview>
- Hooks guide: <https://code.claude.com/docs/en/hooks-guide>
- Commands reference: <https://code.claude.com/docs/en/commands>

Note: the roadmap.sh page renders its topic map interactively, so the learning path above is organized from the official documentation rather than copied from the roadmap's nodes. Versions, commands and plan details change frequently; verify against the docs before relying on any specific flag or requirement.